
Computer Networks Lab

Week 03

Capturing HTTP and HTTPs Traffic using Wireshark

Tools:

Wireshark: Download from <https://www.wireshark.org/download.html>

Web Browser: Chrome/Edge

Text Editor/IDE: VS Code, Notepad++ or any editor to modify HTML

Objectives:

1. Install and configure Wireshark to capture network traffic.
2. Create and use an HTML form to generate HTTP traffic with sensitive data (e.g., username, password, age).
3. Capture and analyze HTTP traffic to observe plaintext data vulnerabilities.
4. Browse a secure website (e.g., Wikipedia) to generate and capture HTTPS traffic, observing encrypted data.
5. Compare HTTP and HTTPS packet details to understand the role of SSL/TLS encryption.
6. Relate findings to real-world applications, such as secure data transmission for web services.

Introduction to Secure Digital Transmissions

In modern computer networks, the security of data transmissions is crucial. Data is transmitted as digital signals over network cables or wireless channels. The efficiency of this transmission depends on multiple factors, including transmission modes, encoding schemes, bandwidth usage, data rate, signal rate, and error detection mechanisms. These elements collectively determine how efficiently and accurately data moves across a network. As users interact with websites, they frequently send and receive sensitive information, such as login credentials, payment details, and personal data. If this information is transmitted insecurely, attackers can intercept and misuse it, leading to data breaches, identity theft, and other cybersecurity threats.

Two widely used protocols for transmitting data over the web are:

- **HTTP (Hypertext Transfer Protocol):** A protocol for transferring web data (e.g., web pages, forms) between clients (browsers) and servers. It operates on port 80 and sends data in **plaintext**, making it vulnerable to interception.
- **HTTPS (Hypertext Transfer Protocol Secure):** An extension of HTTP that uses SSL/TLS for encryption, ensuring data confidentiality, integrity, and authenticity. It operates on port 443, protecting sensitive data (e.g., passwords, credit card details).
- **Wireshark:** An open-source tool for capturing and analyzing network packets, allowing users to inspect protocol details (e.g., HTTP POST requests, HTTPS encrypted data).
- **Packet:** A unit of data transmitted over a network, containing headers (metadata) and payload (actual data).
- **SSL/TLS (Secure Sockets Layer/Transport Layer Security):** Cryptographic protocols that encrypt data, verify server identity, and prevent tampering. TLS is the modern version, used in HTTPS.
- **DNS (Domain Name System):** A system that translates domain names (e.g., google.com) to IP addresses (e.g., 216.58.216.164). Relevant for resolving website addresses in this lab.

This lab will guide you through capturing and analyzing HTTP and HTTPS traffic using **Wireshark**, a popular network protocol analyzer. By submitting form data over HTTP and HTTPS, you will observe the critical differences between plaintext and encrypted transmissions.

Capturing HTTP Traffic using Wireshark

To analyze HTTP traffic, we will use a simple HTML form provided in Google Classroom. This form contains only a **username** and **password** field and submits data using the HTTP protocol. Since HTTP transmits data in plaintext, we will use Wireshark to capture the network traffic and examine how easily sensitive information can be intercepted.

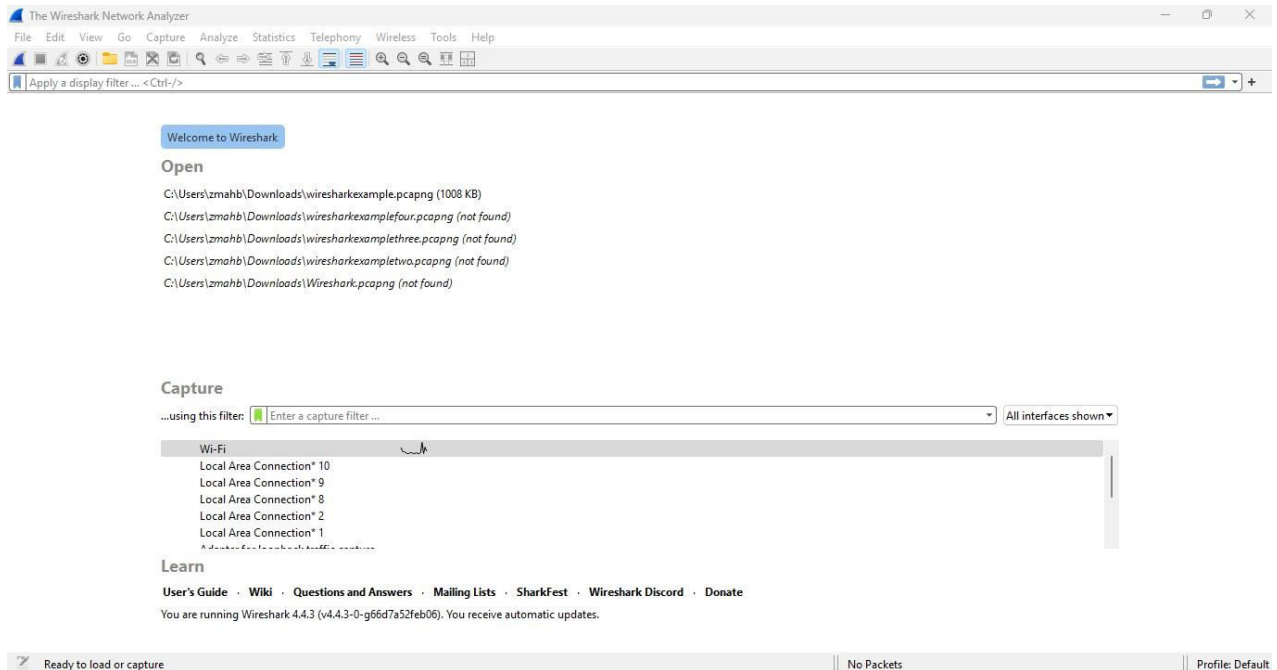
Start by opening **Wireshark** and selecting the appropriate network interface, either **Wi-Fi** or **Ethernet**, depending on how your computer is connected to the internet. Double click on the given options to begin the packet capture. Once started you can stop by clicking the red button next to the blue fin, or restart by clicking on the **Start Capture** button (the blue fin button). Once Wireshark is recording network traffic, open the HTML form in your web browser and enter test credentials for the **username** and **password** fields. Click the **Submit** button to send the form data over the network.

After submitting the form, stop the packet capture in Wireshark. To filter out irrelevant traffic, enter *http* in the filter bar and press **Enter**. This will display only HTTP packets. Look for a **POST request** in the **Info** column, as this is the request that carries the form data. Click on the **POST request** packet to examine its details. In the **Packet Details pane**, expand the **Hypertext**

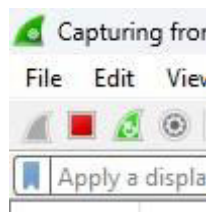
Transfer Protocol (HTTP) section, followed by the **HTML Form URL Encoded** section.

Here, you will be able to see the **username** and **password** in plaintext, exactly as they were entered in the form.

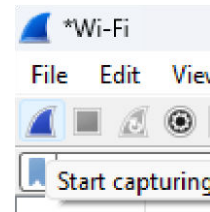
This confirms the fundamental security flaw of HTTP: all transmitted data is unencrypted, meaning that anyone monitoring the network can intercept and read sensitive information such as login credentials. You can also observe some screenshots related to the steps described above, in the sections below.



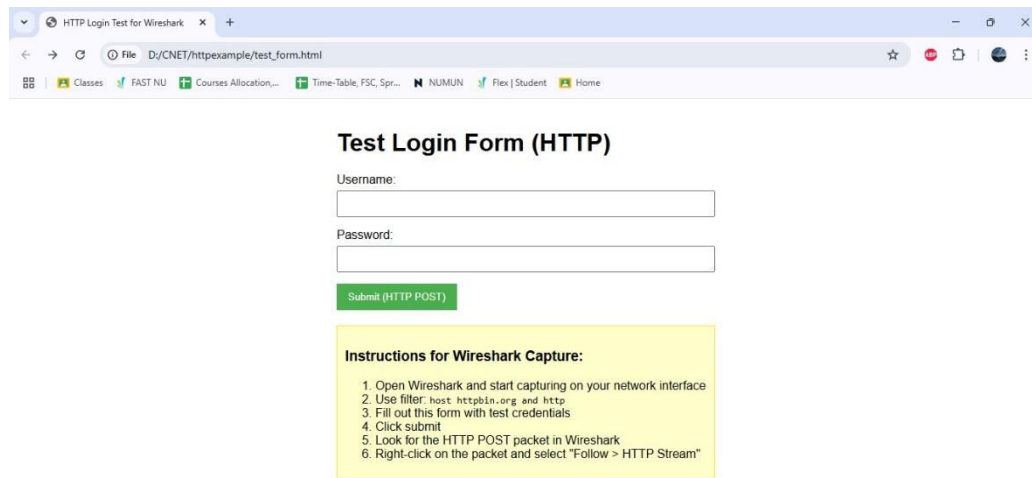
Selecting Wi-Fi interface for capturing packets.



Stop button



Start button



Test Login Form (HTTP)

Username:

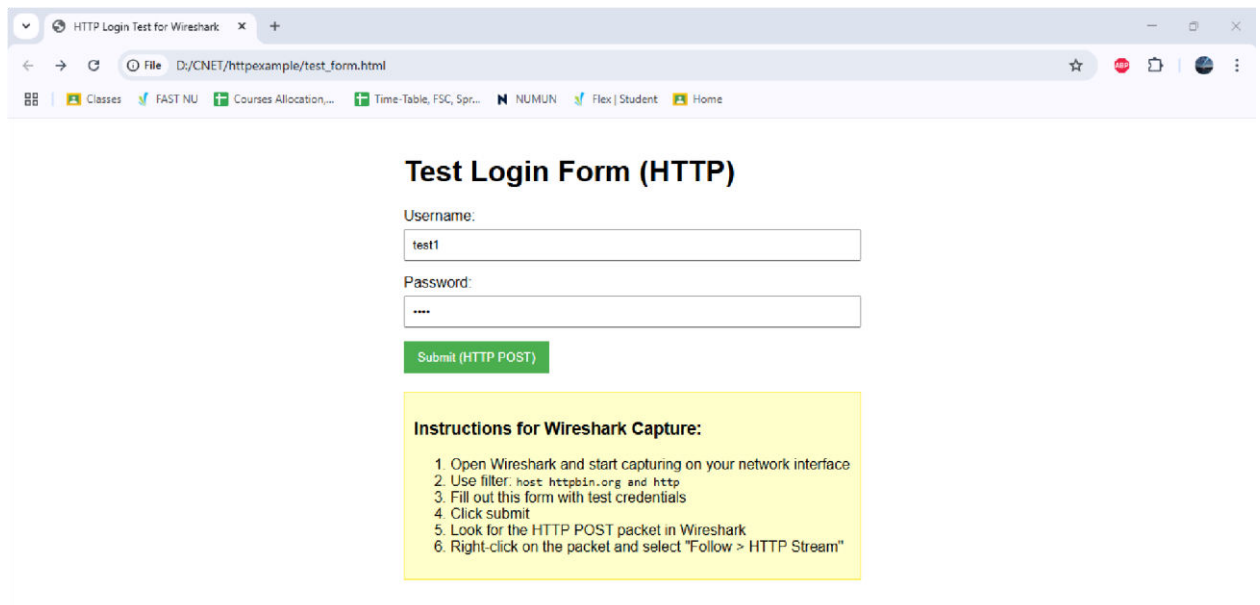
Password:

[Submit \(HTTP POST\)](#)

Instructions for Wireshark Capture:

1. Open Wireshark and start capturing on your network interface
2. Use filter: host httpbin.org and http
3. Fill out this form with test credentials
4. Click submit
5. Look for the HTTP POST packet in Wireshark
6. Right-click on the packet and select "Follow > HTTP Stream"

Accessing the website with the HTML form



Test Login Form (HTTP)

Username:

Password:

[Submit \(HTTP POST\)](#)

Instructions for Wireshark Capture:

1. Open Wireshark and start capturing on your network interface
2. Use filter: host httpbin.org and http
3. Fill out this form with test credentials
4. Click submit
5. Look for the HTTP POST packet in Wireshark
6. Right-click on the packet and select "Follow > HTTP Stream"

Entering insecure credentials

```
httpbin.org/post
Not secure httpbin.org/post
Classes FAST NU Courses Allocation... Time-Table, FSC, Spr... NUMUN Flex | Student Home
Pretty-print
{
  "args": {},
  "data": {},
  "files": {},
  "form": {
    "password": "1234",
    "username": "test1"
  },
  "headers": {
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "en-US,en;q=0.9",
    "Cache-Control": "max-age=0",
    "Content-Length": "28",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "Origin": "null",
    "Upgrade-Insecure-Requests": "1",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36",
    "X-Amzn-Trace-Id": "Root=1-67b07d50-4c8b38cb77eb2a951fe1be2e"
  },
  "json": null,
  "origin": "154.192.8.52",
  "url": "http://httpbin.org/post"
}
```

Response page after insecure credentials have been entered

Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

No.	Time	Source	Destination	Protocol	Length	Info
458	22.173114	192.168.18.17	3.214.119.249	HTTP	622	POST /post HTTP/1.1 (application/x-www-form-urlencoded)
466	22.414310	3.214.119.249	192.168.18.17	HTTP/1.1	922	HTTP/1.1 200 OK, JSON (application/json)

> Frame 458: 622 bytes on wire (4976 bits), 622 bytes captured (4976 bits) on interface \Device\NPF{...}

> Ethernet II, Src: Intel_ff:6d:92 (18:56:00:ff:6d:92), Dst: HuaweiTechno_f7:a7:5a (6c:44:2a:f7:a7:5a)

> Internet Protocol Version 4, Src: 192.168.18.17, Dst: 3.214.119.249

> Transmission Control Protocol, Src Port: 50660, Dst Port: 80, Seq: 1, Ack: 1, Len: 568

> Hypertext Transfer Protocol

> HTML Form URL Encoded: application/x-www-form-urlencoded

0000 6c 44 2a f7 a7 5a 18 56 80 ff 6d 92 08 00 45 00 ID*...Z-V...E...
0010 02 60 69 fe 40 00 00 06 40 11 c0 a8 12 11 03 d6 i...@...@...
0020 77 f9 c5 e4 00 50 d4 47 ea ab c6 2e ba c3 50 18 w...P-G...P...
0030 00 ff ad 9c 00 00 50 4f 53 54 20 2f 70 6f 73 74PO ST /post
0040 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74 3a HTTP/1.1: Host:
0050 20 68 74 74 70 62 69 6e 2e 6f 72 67 0d 0a 43 6f httpbin.org: Co
0060 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 nnection: keep-a
0070 6c 69 76 65 0d 0a 43 6f 6e 74 65 6e 74 2d 4c 65 live: Co ntent-Le
0080 6e 67 74 68 3a 20 32 38 0d 0a 43 61 63 68 65 2d ngth: 28 ..Cache-
0090 43 6f 6e 74 72 6f 6c 3a 20 6d 61 78 2d 61 67 65 Control: max-age
00a0 3d 30 0d 0a 4f 72 69 67 69 6e 3a 20 6e 75 6c 6c => Orig in: null
00b0 0d 0a 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a 20 ..Conten t-Type:
00c0 61 70 70 6c 69 63 61 74 69 6f 6e 2f 78 2d 77 77 applicat ion/x-w
00d0 77 2d 66 6f 72 6d 2d 75 72 6c 65 6e 63 6f 64 65 w-form-u rlencode
00e0 64 0d 0a 55 70 67 72 61 64 65 2d 49 6e 73 65 63 d- Upgra de-Insec
00f0 75 72 65 2d 52 65 71 75 65 73 74 73 3a 20 31 0d ure-Requ ests: 1
0100 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d 6f 7a ..User-Ag ent: Moz
0110 69 6c 6c 61 2f 35 2e 30 20 28 57 69 6e 64 6f 77 illa/5.0 (Wind
0120 73 20 4e 54 20 31 30 2e 30 3b 20 57 69 6e 36 34 s NT 10. 0; Win64

Hypertext Transfer Protocol: Protocol

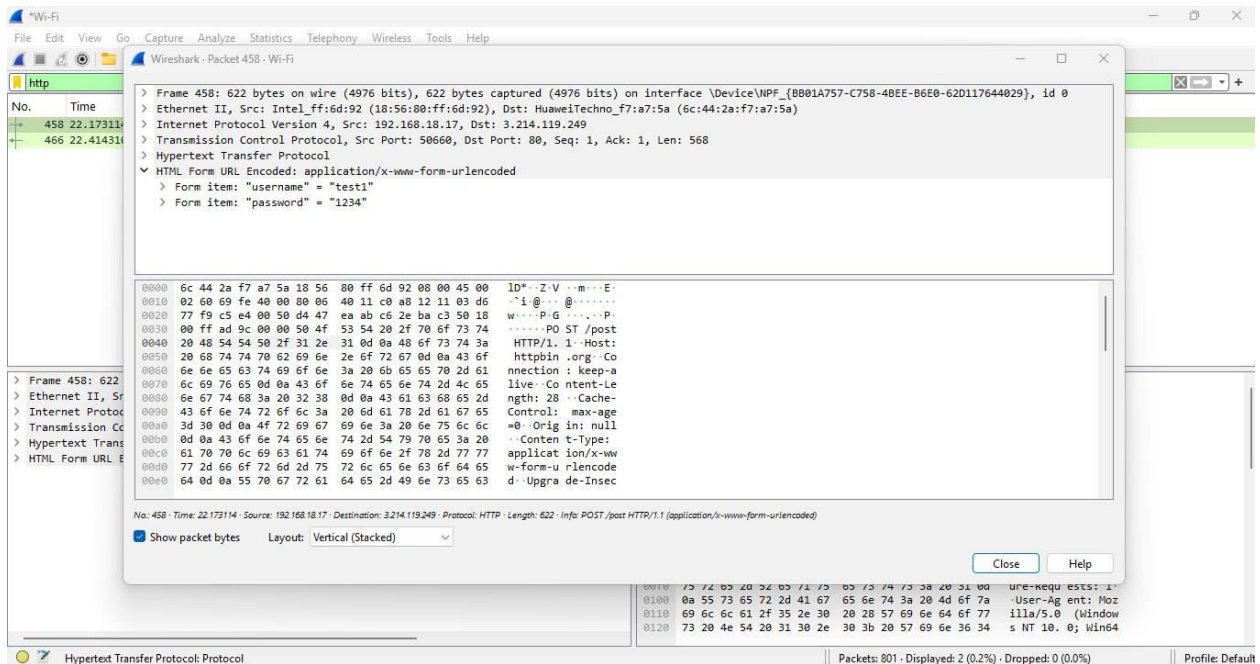
Packets: 801 · Displayed: 2 (0.2%) · Dropped: 0 (0.0%) Profile: Default

Rain showers
Next Thursday

Search

4:41 PM
15-Feb-25

Filtering packets to show HTTP traffic



Opening relevant packet to see details sent through HTTP request

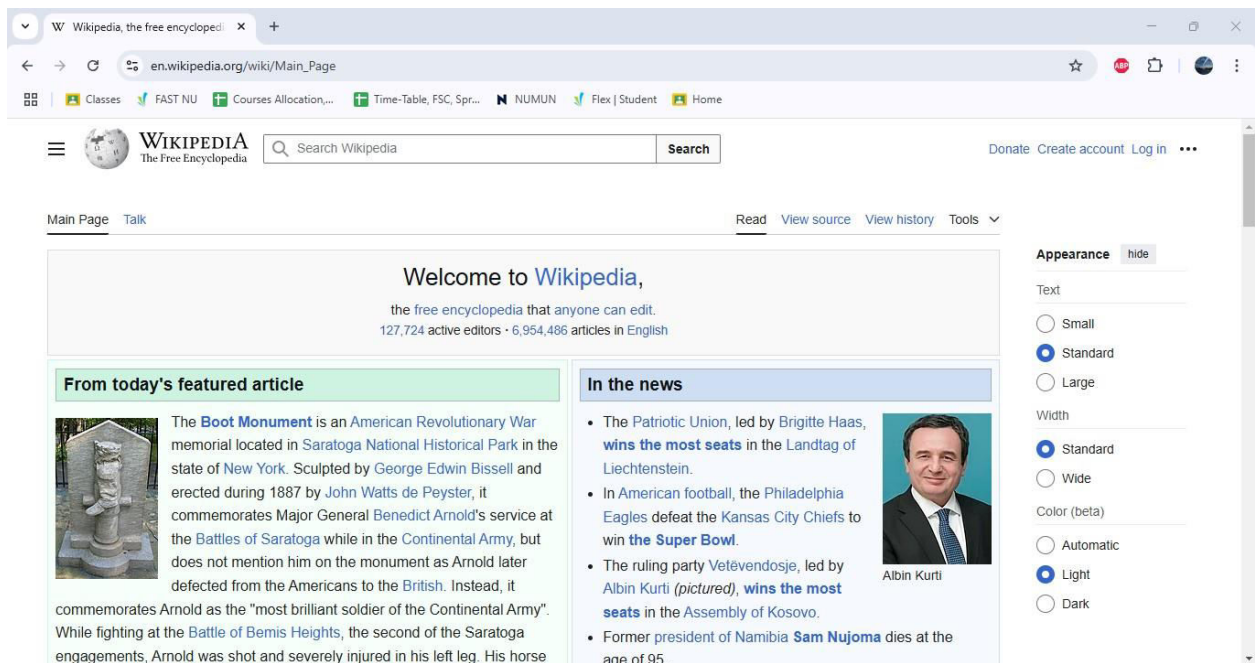
Capturing HTTPS Traffic using Wireshark

To analyze HTTPS traffic, we will visit **Wikipedia** and search for the term "**cheese**", then browse through different pages on the website. Unlike HTTP, HTTPS encrypts all transmitted data using SSL/TLS, making it unreadable even when intercepted. Our goal is to capture HTTPS packets in Wireshark and observe how the encryption prevents us from seeing the actual data being transmitted.

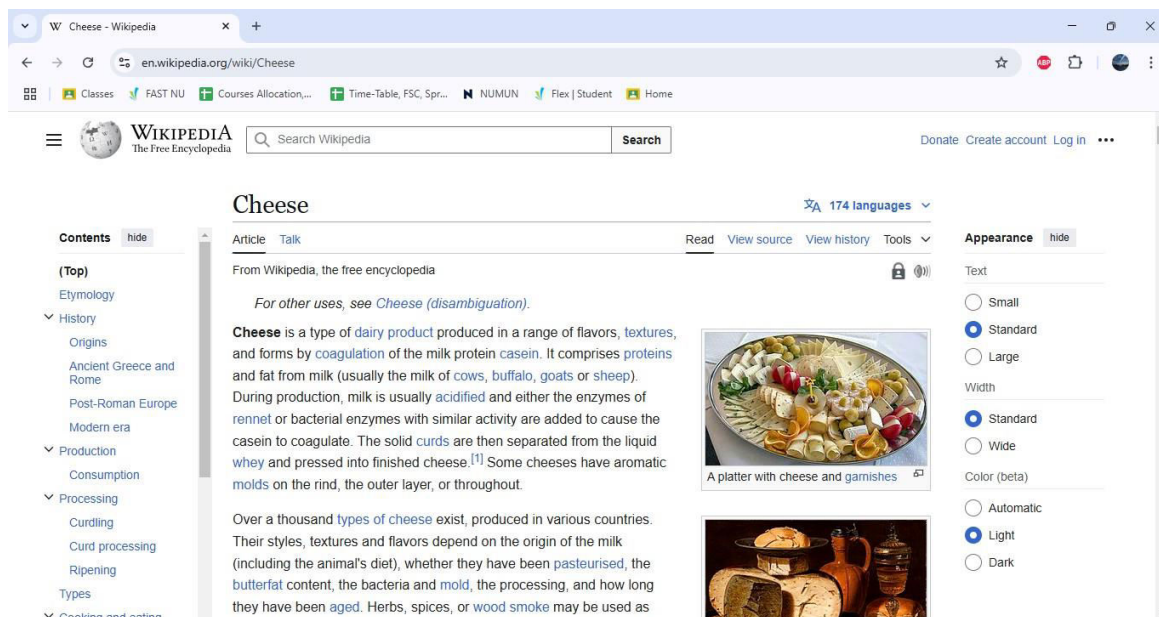
Begin by opening **Wireshark** and selecting the appropriate network interface. Click on the **Start Capture** button to begin recording network traffic. With Wireshark running in the background, open a web browser and navigate to <https://www.wikipedia.org/>. In the Wikipedia search bar, type "**cheese**" and press **Enter**. Browse through various cheese-related articles for a minute to generate sufficient network traffic.

Once you have interacted with the website, stop the packet capture in Wireshark. To isolate HTTPS traffic, apply a filter by entering either `tcp.port == 443` or `ssl` in the filter bar and pressing **Enter**. Unlike HTTP, you will not see clearly readable text in the captured packets. Instead, locate a packet labeled **Application Data** under the **TLSv1.2** or **TLSv1.3** protocol. Clicking on this packet will reveal an **Encrypted Application Data** section in the **Packet Details** pane, indicating that the actual content of the transmission is securely encrypted.

This confirms that HTTPS successfully protects sensitive information by encrypting it before transmission. Unlike HTTP, where data is sent in plaintext, HTTPS ensures that even if network traffic is intercepted, the actual content remains hidden from unauthorized parties. This encryption is what makes HTTPS the preferred choice for secure digital communications, safeguarding user privacy and protecting sensitive data from cyber threats. You can also observe some screenshots related to the steps described above, in the sections below.

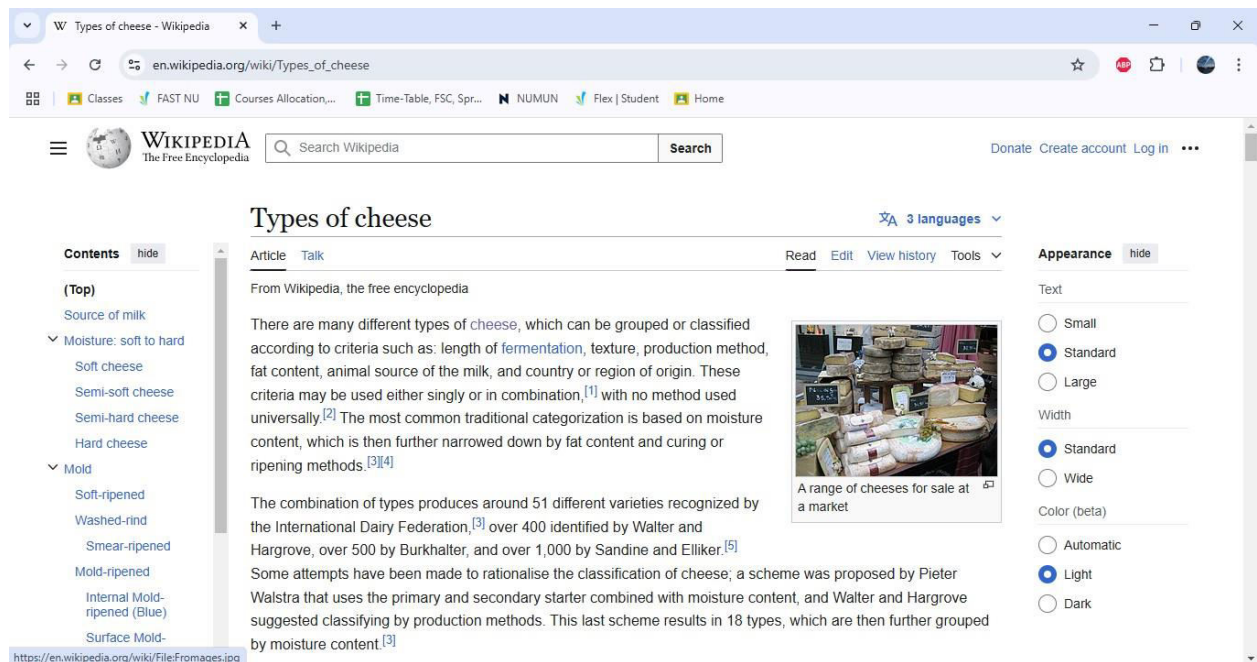


Going to Wikipedia while Wireshark runs in the background



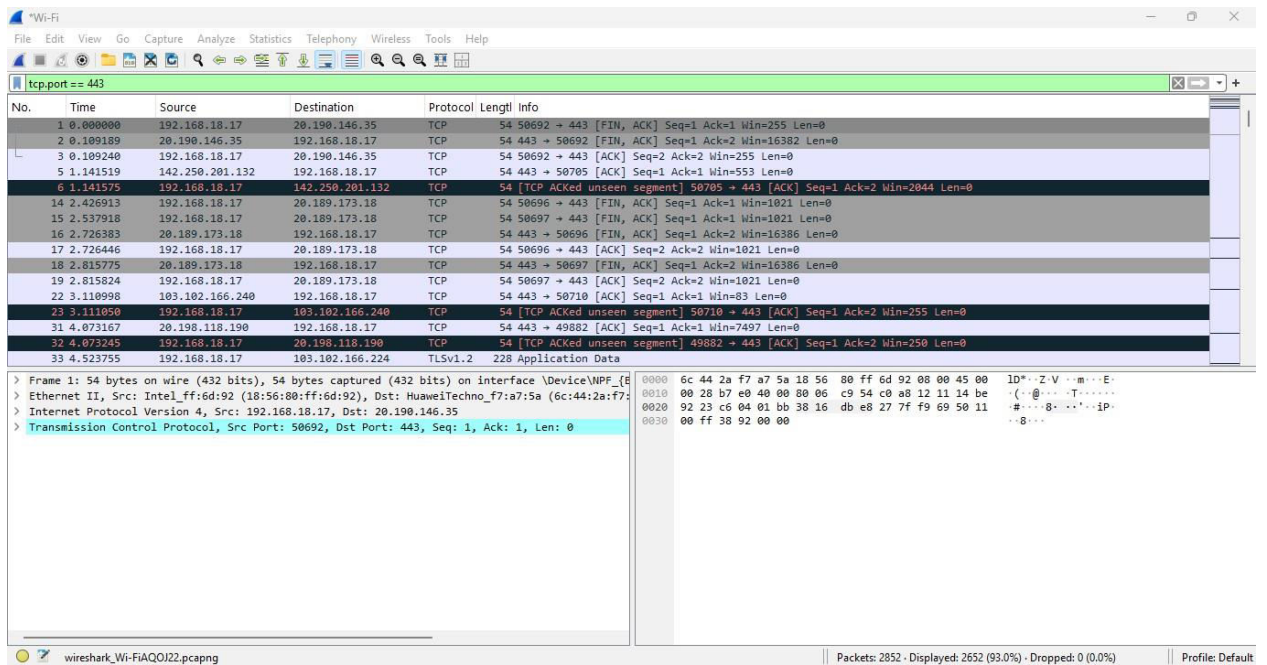
The screenshot shows the Wikipedia page for "Cheese". The page is in English and has 174 languages available. The main content area includes a definition of cheese, a list of types of cheese, and a list of related topics. The page is displayed in a standard layout with a sidebar on the left and a main content area on the right.

Results after searching “cheese”



The screenshot shows the Wikipedia page for "Types of cheese". The page is in English and has 3 languages available. The main content area includes a definition of types of cheese, a list of types of cheese, and a list of related topics. The page is displayed in a standard layout with a sidebar on the left and a main content area on the right.

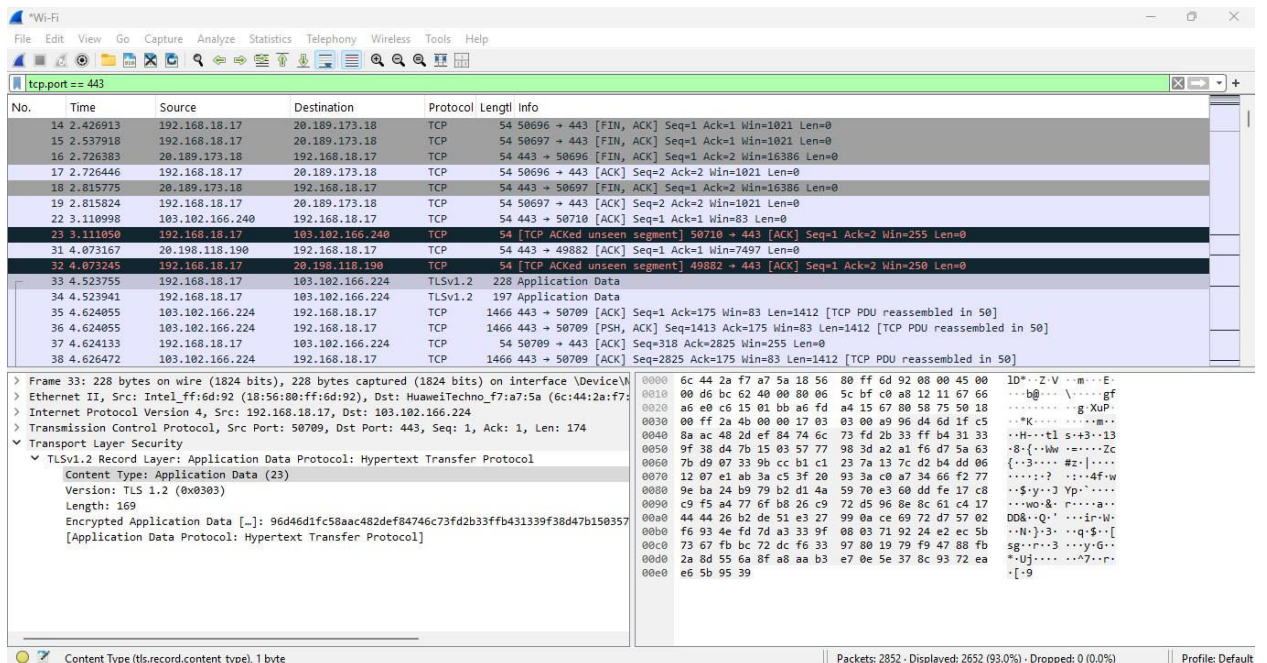
Browsing through different cheese related pages.



Wireshark capture showing traffic on tcp.port == 443. The packet list displays several TCP segments, including a FIN segment (Seq=1, Ack=1, Win=255, Len=0) and an ACK segment (Seq=2, Ack=2, Win=16382, Len=0). The packet details pane shows the selected packet (No. 33) as a TLSv1.2 record, indicating encrypted data. The packet bytes pane displays the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.18.17	20.190.146.35	TCP	54	50692 → 443 [FIN, ACK] Seq=1 Ack=1 Win=255 Len=0
2	0.109189	20.190.146.35	192.168.18.17	TCP	54	443 → 50692 [FIN, ACK] Seq=1 Ack=2 Win=16382 Len=0
3	0.109240	192.168.18.17	20.190.146.35	TCP	54	50692 → 443 [ACK] Seq=2 Ack=1 Win=255 Len=0
5	1.141519	142.250.201.132	192.168.18.17	TCP	54	443 → 50705 [ACK] Seq=1 Ack=1 Win=553 Len=0
6	1.141575	192.168.18.17	142.250.201.132	TCP	54	[TCP ACKed unseen segment] 50705 → 443 [ACK] Seq=1 Ack=2 Win=2044 Len=0
14	2.426913	192.168.18.17	20.189.173.18	TCP	54	50696 → 443 [FIN, ACK] Seq=1 Ack=1 Win=1021 Len=0
15	2.537918	192.168.18.17	20.189.173.18	TCP	54	50697 → 443 [FIN, ACK] Seq=1 Ack=1 Win=1021 Len=0
16	2.726383	20.189.173.18	192.168.18.17	TCP	54	443 → 50696 [FIN, ACK] Seq=1 Ack=2 Win=16386 Len=0
17	2.726446	192.168.18.17	20.189.173.18	TCP	54	50696 → 443 [ACK] Seq=2 Ack=2 Win=1021 Len=0
18	2.815775	20.189.173.18	192.168.18.17	TCP	54	443 → 50697 [FIN, ACK] Seq=1 Ack=2 Win=16386 Len=0
19	2.815824	192.168.18.17	20.189.173.18	TCP	54	50697 → 443 [ACK] Seq=2 Ack=2 Win=1021 Len=0
22	3.110998	103.102.166.240	192.168.18.17	TCP	54	443 → 50710 [ACK] Seq=1 Ack=1 Win=83 Len=0
23	3.111050	192.168.18.17	103.102.166.240	TCP	54	[TCP ACKed unseen segment] 50710 → 443 [ACK] Seq=1 Ack=2 Win=255 Len=0
31	4.073167	20.198.118.190	192.168.18.17	TCP	54	443 → 49882 [ACK] Seq=1 Ack=1 Win=7497 Len=0
32	4.073245	192.168.18.17	20.198.118.190	TCP	54	[TCP ACKed unseen segment] 49882 → 443 [ACK] Seq=1 Ack=2 Win=250 Len=0
33	4.523755	192.168.18.17	103.102.166.224	TLSv1.2	228	Application Data

Applying `tcp.port == 443` (the port used for HTTPS traffic) to see HTTPS traffic



Wireshark capture showing traffic on tcp.port == 443. The packet list displays several TCP segments, including a FIN segment (Seq=1, Ack=1, Win=1021, Len=0) and an ACK segment (Seq=2, Ack=2, Win=16386, Len=0). The packet details pane shows the selected packet (No. 33) as a TLSv1.2 record, indicating encrypted data. The packet bytes pane displays the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
14	2.426913	192.168.18.17	20.189.173.18	TCP	54	50696 → 443 [FIN, ACK] Seq=1 Ack=1 Win=1021 Len=0
15	2.537918	192.168.18.17	20.189.173.18	TCP	54	50697 → 443 [FIN, ACK] Seq=1 Ack=1 Win=1021 Len=0
16	2.726383	20.189.173.18	192.168.18.17	TCP	54	443 → 50696 [FIN, ACK] Seq=1 Ack=2 Win=16386 Len=0
17	2.726446	192.168.18.17	20.189.173.18	TCP	54	50696 → 443 [ACK] Seq=2 Ack=2 Win=1021 Len=0
18	2.815775	20.189.173.18	192.168.18.17	TCP	54	443 → 50697 [FIN, ACK] Seq=1 Ack=2 Win=16386 Len=0
19	2.815824	192.168.18.17	20.189.173.18	TCP	54	50697 → 443 [ACK] Seq=2 Ack=2 Win=1021 Len=0
22	3.110998	103.102.166.240	192.168.18.17	TCP	54	443 → 50710 [ACK] Seq=1 Ack=1 Win=83 Len=0
23	3.111050	192.168.18.17	103.102.166.240	TCP	54	[TCP ACKed unseen segment] 50710 → 443 [ACK] Seq=1 Ack=2 Win=255 Len=0
31	4.073167	20.198.118.190	192.168.18.17	TCP	54	443 → 49882 [ACK] Seq=1 Ack=1 Win=7497 Len=0
32	4.073245	192.168.18.17	20.198.118.190	TCP	54	[TCP ACKed unseen segment] 49882 → 443 [ACK] Seq=1 Ack=2 Win=250 Len=0
33	4.523755	192.168.18.17	103.102.166.224	TLSv1.2	228	Application Data
34	4.523941	192.168.18.17	103.102.166.224	TLSv1.2	197	Application Data
35	4.624855	103.102.166.224	192.168.18.17	TCP	1466	443 → 50709 [ACK] Seq=1 Ack=175 Win=83 Len=1412 [TCP PDU reassembled in 50]
36	4.624855	103.102.166.224	192.168.18.17	TCP	1466	443 → 50709 [PSH, ACK] Seq=1413 Ack=175 Win=83 Len=1412 [TCP PDU reassembled in 50]
37	4.624133	192.168.18.17	103.102.166.224	TCP	54	50709 → 443 [ACK] Seq=318 Ack=2825 Win=255 Len=0
38	4.626472	103.102.166.224	192.168.18.17	TCP	1466	443 → 50709 [ACK] Seq=2825 Ack=175 Win=83 Len=1412 [TCP PDU reassembled in 50]

Observing encrypted application data

Lab Tasks

Task 1: Analyzing HTTP Traffic (10 marks)

Using the code given in the google classroom, you will generate HTTP traffic and show insecurely sent data clearly displayed on Wireshark. For this, download the code from the GCR and, using any IDE, add a field for age in the form and add your **roll number and name** in place of `<h1> </h1>`. Then you will follow these steps, **recording each step by taking screenshots of it and attaching them to a pdf file/word file:**

Start the Wireshark Capture

1. Open **Wireshark**.
2. Select the **network interface** that your computer uses for internet access (e.g., Wi-Fi or Ethernet).
3. Click **Start Capture**.

Submit the Form

4. Open **test_form.html** in a web browser.
5. Enter the following details in the form:
 - **Username:** Your roll number
 - **Password:** Any random password (e.g; 1234)
 - **Age:** Any number
6. Click **Submit**.

Filter and Analyze HTTP Packets

7. Stop the Wireshark capture.
8. In the **Wireshark Filter Bar**, type `http` and press **Enter** to isolate HTTP traffic.
9. Locate the **POST** request packet that contains the submitted form data.
10. Click on the packet and expand the **HTML Form URL Encoded** section in the lower panel.
11. Take a **screenshot** showing the captured username, password, and age in plaintext.

Also, **answer** the following question:

Q1. Why is HTTP considered insecure for transmitting sensitive data? Explain the risks of plaintext transmission and how attackers can exploit it.

Task 2: Analyzing HTTPS Traffic (10 marks)

By navigating through different pages on Wikipedia, you will generate HTTPS traffic that you will then capture and analyze to see how encryption works. For this, you will go to <https://www.wikipedia.org/> and browse through several pages related to tomatoes. The steps you will follow, **recording each step by taking screenshots of it and attaching them to a pdf file/word file**, are:

Start a Wireshark Capture

1. Open **Wireshark** and start a new packet capture session (details also given in task 1).

Visit Wikipedia and look for a page related to Tomatoes

2. Open your web browser and navigate to [Wikipedia's Tomato page](#).
3. Browse the page for up to 1 minute to generate network traffic.

Filtering and Analyzing HTTPS Packets

4. Stop the Wireshark capture.
5. In the **Wireshark Filter Bar**, type `tcp.port == 443` or `ssl` and press **Enter** to isolate HTTPS traffic.
6. Locate a packet labeled **Application Data**—this contains encrypted content.
7. Take a **screenshot** of the packet details showing that the data is encrypted.

Also, **answer** the following questions:

Q1. What differences do you notice between HTTP and HTTPS traffic? Compare the packet details for HTTP vs. HTTPS and note how the data is secured.

Q2. Why is HTTPS considered more secure than HTTP? Explain the role of SSL/TLS encryption in preventing data interception and tampering.

Task 3: Capturing and Understanding Packet Transmission (10 marks)

The first step in analyzing digital transmission is to capture network packets and examine their transmission characteristics. You will capture **ICMP (ping) packets** to observe how data is transmitted serially over the network.

For this you will:

Open **Wireshark** and select the **active network interface** (Ethernet or Wi-Fi). Start a packet

capture session by clicking the **Start Capture button**. Open the command prompt on your system and run this: `ping -n 5 www.google.com`. Which will send five **ICMP echo request** packets to Google's server. After the ping operation completes, **stop the packet capture** in Wireshark. In the Wireshark **filter bar**, enter the `icmp` filter to isolate ICMP packets.

Select an **ICMP request packet** and examine the **Frame Details**. Record the following information on a document:

1. Packet arrival time (timestamp) [any packet]
2. Frame length (bytes) [any packet]
3. Time difference between consecutive packets (inter-packet delay) [first and last request]

Based off this information, answer the following questions:

Q1. How frequently are packets being sent?

Q2. What is the **average packet size**, and how does it compare to other traffic types? (hint: you will have to generate different traffic)

Q3. What transmission mode is used?

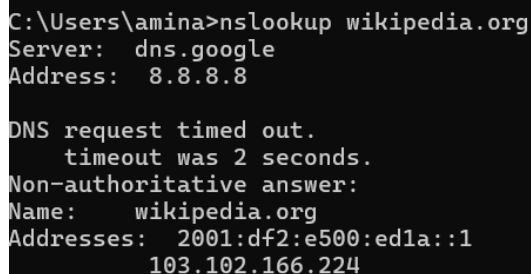
Task 4: Analysing DNS Traffic (10 marks)

Capture and analyze DNS traffic to understand how domain names (e.g., www.google.com) resolve to IP addresses (e.g., 142.250.202.238).

1. Open **Wireshark** select your interface and **start capturing** network traffic.
2. Open Command Prompt and run:

```
nslookup wikipedia.org
```

3. Note the IP Address 103.102.166.224 and capture the screenshot as below:



```
C:\Users\amina>nslookup wikipedia.org
Server:  dns.google
Address:  8.8.8.8

DNS request timed out.
  timeout was 2 seconds.
Non-authoritative answer:
Name:     wikipedia.org
Addresses: 2001:df2:e500:ed1a::1
          103.102.166.224
```

4. Stop Wireshark capture.
5. Filter and Analyze DNS packets

- In the filter bar, type **dns** and press Enter.
- Find a DNS Query packet for www.wikipedia.org and a DNS Response packet with the IP.
- Click the response packet, expand Domain Name System (response) > Answers. Verify the IP (e.g., 103.102.166.224).
- **Screenshot:** Capture Wireshark showing the DNS response (label: “Figure: DNS Response Packet”).

6. Answer the Question:

Q1: Why is DNS critical for HTTP/HTTPS communication, and how does it relate to secure data transmission?

Submission Guidelines:

- Submit a single PDF document **AND THE HTML PAGE** containing all tasks to the GCR. (**no wireshark files**)
- Ensure that all screenshots are clear and labeled appropriately.
- Include your name, roll number, and lab number inside the **first page** of the document.
- Name the submitted document as follows: SectionLetter_RollNumber_LabNumber, for example: **J_i221234_Lab1**. Rename the HTML file as **rollnumber_name.html**.
- In case a demo is conducted, there will be deductions based on your answers to the questions asked.

NOTE: You must show clear screenshots of every step you take. Marks will be deducted for this mistake and in some cases you may be awarded 0 (this applies to all tasks).

Marking Criteria (45 marks):

Either 0 [not achieved], half [partial achieved] or full [achieved]

Task 1: Analyzing HTTP Traffic (10 Marks)

1. Modification of HTML Form (2 Marks)

- Successfully added the "Age" field in the form and also their name and roll number - **ZERO OR HALF MAX FOR NOT DISPLAYING ROLL NUMBER/NAME.**

2. Wireshark Capture & HTTP Packet Analysis (3 Marks)

- Captured HTTP traffic correctly, filtered for HTTP packets, and located the POST request with form data.

3. Screenshots (3 Marks)

- Provided clear, labeled screenshots of each step, including the captured username, password, and age in plaintext.

4. Answer to Q1 (2 Marks)

- Clearly explained why HTTP is insecure, the risks of plaintext transmission, and how attackers can exploit it.

Task 2: Analyzing HTTPS Traffic (10 Marks)

1. Wireshark Capture & HTTPS Packet Analysis (3 Marks)

- Captured HTTPS traffic correctly, filtered for HTTPS packets, and located encrypted Application Data.

2. Browsing Wikipedia Pages (2 Marks)

- Visited and browsed Wikipedia pages related to tomatoes to generate HTTPS traffic.

3. Screenshots (3 Marks)

- Provided clear, labeled screenshots of each step, including encrypted Application Data.

4. Answers to Q1 & Q2 (2 Marks)

- Explained the differences between HTTP and HTTPS, and the role of SSL/TLS in securing data.

Task 3: Capturing and Understanding Packet Transmission (10 Marks)

1. Packet Capture & Filtering (4 Marks)

- 2 mark for successfully capturing ICMP packets in Wireshark.
- 2 mark for applying the icmp filter and selecting a request packet.

2. Data Recording (4 Marks)

- 2 mark for correctly recording packet arrival time, frame length, and inter-packet delay.
- 2 mark for analyzing inter-packet delay trends.

3. Answering Questions (2 Mark)

- 2 mark for accurately answering Q1–Q3 based on observations.

Task 4: Analyzing DNS Traffic (10 Marks)

1. Packet Capture & Filtering (4 Marks)

- Capture and filter DNS traffic.

2. Screenshots and captions (4 Marks)

- Necessary Screenshots

3. Answering Questions (2 Mark)

- 2 marks for accurately answering Q1 based on observations.

Submission Guidelines (5 Marks)

1. PDF Document Submission and HTML file (2 Marks) - **NO ZIP files.**

- Single, clear, and well-organized PDF document with all tasks and screenshots.
- HTML file containing their roll number and name (inside the file).

2. File Naming (1 Mark)

- PDF and form files correctly named as per the guidelines.

3. Details on the First Page (1 Mark)

- Name, roll number, section and lab number included.

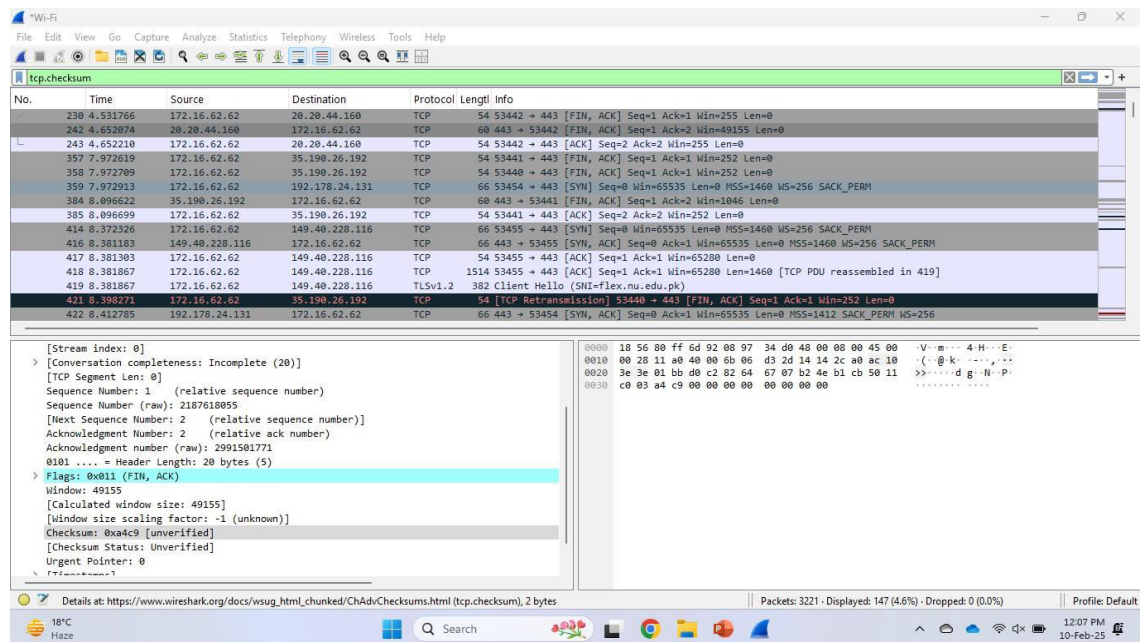
4. Clear and Professional Presentation (1 Mark)

- Screenshots, labels, and descriptions are neat and easy to understand.

Demo:

- If student cannot answer at all -5 marks.
- If student can answer somewhat -2 marks. - Correct answers results in no deductions.

Sample screenshot:



The screenshot displays the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The main window is titled "Wi-Fi" and shows a list of captured packets. The selected packet is a TCP segment (No. 421) with a sequence number of 53440 and an acknowledgment number of 443. The packet details pane on the right shows the structure of the TCP segment, including the sequence number, acknowledgment number, window size, and flags. The packet bytes pane on the right shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
230	4.531766	172.16.62.62	20.20.44.160	TCP	54	53442 → 443 [FIN, ACK] Seq=1 Ack=1 Win=255 Len=0
242	4.652074	20.20.44.160	172.16.62.62	TCP	60	443 → 53442 [FIN, ACK] Seq=1 Ack=2 Win=49155 Len=0
243	4.652210	172.16.62.62	20.20.44.160	TCP	54	53442 → 443 [ACK] Seq=2 Ack=2 Win=255 Len=0
357	7.972619	172.16.62.62	35.190.26.192	TCP	54	53441 → 443 [FIN, ACK] Seq=1 Ack=1 Win=252 Len=0
358	7.972709	172.16.62.62	35.190.26.192	TCP	54	53440 → 443 [FIN, ACK] Seq=1 Ack=1 Win=252 Len=0
359	7.972913	172.16.62.62	192.178.24.131	TCP	66	53454 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
384	8.096622	35.190.26.192	172.16.62.62	TCP	60	443 → 53441 [FIN, ACK] Seq=1 Ack=2 Win=1846 Len=0
385	8.096699	172.16.62.62	35.190.26.192	TCP	54	53441 → 443 [ACK] Seq=2 Ack=2 Win=252 Len=0
414	8.372326	172.16.62.62	149.40.228.116	TCP	66	53455 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
416	8.381183	149.40.228.116	172.16.62.62	TCP	66	443 → 53455 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
417	8.381303	172.16.62.62	149.40.228.116	TCP	54	53455 → 443 [ACK] Seq=1 Ack=1 Win=65200 Len=0
418	8.381867	172.16.62.62	149.40.228.116	TCP	1514	53455 → 443 [ACK] Seq=1 Ack=1 Win=65200 Len=1460 [TCP PDU reassembled in 419]
419	8.381867	172.16.62.62	149.40.228.116	TLShv1.2	382	Client Hello (SHA=flex.nu.edu.pk)
421	8.398271	172.16.62.62	35.190.26.192	TCP	54	[TCP Retransmission] 53440 → 443 [FIN, ACK] Seq=1 Ack=1 Win=252 Len=0
422	8.412785	192.178.24.131	172.16.62.62	TCP	66	443 → 53454 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256

Details at: https://www.wireshark.org/docs/wsug_html_chunked/ChAdvChecksums.html (tcp.checksum), 2 bytes

Packets: 3221 · Displayed: 147 (4.6%) · Dropped: 0 (0.0%)

Profile: Default

18°C Haze

12:07 PM 10-Feb-25